Teoria współbieżności

Sprawozdanie z laboratorium 2

Adrian Mikoda

19.10.2026 r.

Spis treści

1. Dane techniczne 2
2. Treści zadań 2
   2.1. Zadanie 1 2
   2.2. Zadanie 2 2
   2.3. Zadanie 3 2
3. Realizacja ćwiczenia 2
   3.1. Zadanie 1 – semafor binarny 2
   3.2. Zadanie 2 – `if` kontra `while` 6
   3.3. Zadanie 3 – semafor licznikowy 7
4. Wyniki doświadczenia 8
   4.1. Zadanie 1 i 2 8
   4.2. Wyjaśnienie teoretyczne 9
   4.3. Zadanie 3 10
5. Wnioski 10
Bibliografia 11

1. Dane techniczne

- System operacyjny: `Windows 11 Home`
- Procesor: `AMD Ryzen 5 7530U (2.0 - 4.5 GHz)`
- Pamięć RAM: `16GB LPDDR4 (2133 MHz)`
- SDK: `Microsoft OpenJDK 25.0.4`
- Opcje JVM: `-Xint` (wyłączony kompilator JIT)
- Wizualizacja wyników: `Python 3.13.5`
- Użyte biblioteki: `matplotlib 3.11.2, numpy 2.5.3`

2. Treści zadań

2.1. Zadanie 1

Zaimplementować semafor binarny za pomocą metod `wait` i `notify` oraz użyć go do synchronizacji programu *Wyścig*.

2.2. Zadanie 2

Pokazać, że do implementacji semafora za pomocą metod `wait` i `notify` nie wystarczy instrukcja `if`, tylko potrzebna jest pętla `while`. Wyjaśnić teoretycznie dlaczego i potwierdzić eksperymentem w praktyce. (Wskazówka: rozważyć dwie kolejki: oczekującą na wejście do monitora obiektu oraz kolejkę związaną z instrukcją `wait`; rozważyć, kto i kiedy jest budzony oraz kiedy następuje wyścig).

2.3. Zadanie 3

Zaimplementować semafor licznikowy (ogólny) za pomocą semaforów binarnych. Czy semafor binarny jest szczególnym przypadkiem semafora ogólnego?

3. Realizacja ćwiczenia

3.1. Zadanie 1 – semafor binarny

Klasy `Counter`, `IThread`, `DThread`, `IThreadSemaphore` oraz `DThreadSemaphore` zaimplementowano w pliku `Race.java` (fragment kodu 1). Semafor binarny zaimplementowano w klasie `Semaphore` (fragment kodu 2). Przechowuje ona pole logiczne `open` oznaczające, czy semafor jest otwarty. Metoda `P()` jest zadeklarowana jako `synchronized`; jeśli semafor jest zamknięty, wątek wywołuje `wait()`, a po obudzeniu zamyka semafor (`open = false`). Metoda `V()` otwiera semafor (`open = true`) i wywołuje `notify()`.

Semafor użyto do ochrony operacji `inc()` oraz `dec()` na wspólnym liczniku w programie *Wyścig* (fragment kodu 1). Każdy wątek przed operacją na liczniku wywołuje `P()`, a po niej `V()`. Eksperyment wykonano dla 4 wątków (2 zwiększające i 2 zmniejszające licznik), każdy wykonywał $n = 10000$ operacji. Eksperyment powtórzono 100 razy dla trzech wariantów: bez synchronizacji, z semaforem z instrukcją `if` oraz z semaforem z pętlą `while` na (rysunku).

```java
class Counter {
    private int _val;
    public Counter(int n) { _val = n; }
    public void inc() { _val++; }
    public void dec() { _val--; }
    public int value() { return _val; }
}

class IThread extends Thread {
    Counter counter;
    int n;

    public IThread(Counter counter, int n) {
        this.counter = counter;
        this.n = n;
    }

    @Override
    public void run() {
        for (int i = 0; i < n; i++) { counter.inc(); }
    }
}


class DThread extends Thread {
    Counter counter;
    int n;

    public DThread(Counter counter, int n) {
        this.counter = counter;
        this.n = n;
    }

    @Override
    public void run() {
        for (int i = 0; i < n; i++) { counter.dec(); }
    }
}

class IThreadSemaphore extends IThread {
    Semaphore semaphore;

    public IThreadSemaphore(Counter counter, int n, Semaphore semaphore) {
        super(counter, n);
        this.semaphore = semaphore;
    }

    @Override
    public void run() {
        for (int i = 0; i < n; i++) {
            semaphore.P();
            counter.inc();
            semaphore.V();
        }
    }
}


class DThreadSemaphore extends DThread {
    Semaphore semaphore;
```

```
61
62      public DThreadSemaphore(Counter counter, int n, Semaphore semaphore) {
63          super(counter, n);
64          this.semaphore = semaphore;
65      }
66
67      @Override
68      public void run() {
69          for (int i = 0; i < n; i++) {
70              semaphore.P();
71              counter.dec();
72              semaphore.V();
73          }
74      }
75  }
76
77  public class Race {
78      private static int getCounterValue(Counter counter, DThread dThread, IThread
iThread) {
79          dThread.start();
80          iThread.start();
81          try {
82              dThread.join();
83              iThread.join();
84          } catch (InterruptedException e) {
85              System.out.println("Thread interrupted");
86          }
87
88          return counter.value();
89      }
90
91      public static void main(String[] args) throws InterruptedException {
92          String path_without_synchronization = "../race_without_synchronization.txt";
93          String path_semaphore_if = "../race_semaphore_if.txt";
94          String path_semaphore_while = "../race_semaphore_while.txt";
95
96          try {
97              Files.deleteIfExists(Paths.get(path_without_synchronization));
98              Files.deleteIfExists(Paths.get(path_semaphore_if));
99              Files.deleteIfExists(Paths.get(path_semaphore_while));
100         } catch (Exception e) {
101             e.printStackTrace();
102         }
103
104         for (int i = 0; i < 100; i++) {
105             Counter counter = new Counter(0);
106             DThread dThread = new DThread(counter, 10000);
107             IThread iThread = new IThread(counter, 10000);
108             String content = getCounterValue(counter, dThread, iThread) +
System.lineSeparator();
109             try {
110                 Files.write(Paths.get(path_without_synchronization),
content.getBytes(), StandardOpenOption.CREATE, StandardOpenOption.APPEND);
111             } catch (Exception e) {e.printStackTrace();}
112         }
113
114         for (int i = 0; i < 100; i++) {
115             Counter counter = new Counter(0);
116             SemaphoreIf semaphoreIf = new SemaphoreIf();
117             DThreadSemaphore dThreadSemaphoreIf = new DThreadSemaphore(counter, 10000,
semaphoreIf);
```

```
118             IThreadSemaphore iThreadSemaphoreIf = new IThreadSemaphore(counter, 10000,
semaphoreIf);
119             String content = getCounterValue(counter, dThreadSemaphoreIf,
iThreadSemaphoreIf) + System.lineSeparator();
120             try {
121                 Files.write(Paths.get(path_semaphore_if), content.getBytes(),
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
122             } catch (Exception e) {e.printStackTrace();}
123         }
124
125         for (int i = 0; i < 100; i++) {
126             Counter counter = new Counter(0);
127             Semaphore semaphore = new Semaphore();
128             DThreadSemaphore dThreadSemaphore = new DThreadSemaphore(counter, 10000,
semaphore);
129             IThreadSemaphore iThreadSemaphore = new IThreadSemaphore(counter, 10000,
semaphore);
130             String content = getCounterValue(counter, dThreadSemaphore,
iThreadSemaphore) + System.lineSeparator();
131             try {
132                 Files.write(Paths.get(path_semaphore_while), content.getBytes(),
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
133             } catch (Exception e) {e.printStackTrace();}
134         }
135     }
136 }
```

Fragment kodu 1: Plik `Race.java`: klasy `Counter`, `IThread`, `DThread`, `IThreadSemaphore` oraz `DThreadSemaphore`

```
1  class Semaphore {
2      private boolean _state = true;
3      private int _waiting = 0;
4
5      public Semaphore() {}
6
7      public synchronized void P() {
8          while (!_state) {
9               _waiting++;
10              try { wait(); }
11              catch (InterruptedException e) {
12                  e.printStackTrace();
13              };
14              _waiting--;
15          }
16          _state = false;
17     }
18
19     public synchronized void V() {
20         if (_waiting > 0) {
21             _state = true;
22             notify();
23         }
24         else {_state = true;}
25     }
26 }
27
```

Fragment kodu 2: Plik `Semaphore.java`: klasa `Semaphore`

3.2. Zadanie 2 – `if` kontra `while`

Klasa `SemaphoreIf` (fragment kodu 3) różni się od `Semaphore` (fragment kodu 2) tym, że w metodzie `P()` zamiast pętli `while (!open)` użyto instrukcji `if (!open)`. Przeprowadzono te same 100 powtórzeń co w zadaniu 1.

```
1  class SemaphoreIf extends Semaphore {
2      private boolean _state = true;
3      private int _waiting = 0;
4
5      public SemaphoreIf() {}
6
7      @Override
8      public synchronized void P() {
9          if (!_state) {
10             _waiting++;
11             try { wait(); }
12             catch (InterruptedException e) {
13                 e.printStackTrace();
14             };
15             _waiting--;
16         }
17         _state = false;
18     }
19
20     @Override
21     public synchronized void V() {
22         if (_waiting > 0) {
23             _state = true;
24             notify();
25         }
26         else {_state = true;}
27     }
28 }
29
```

Fragment kodu 3: Plik `SemaphoreIf.java`: klasa `SemaphoreIf`

Rysunki oraz statystyki eksperymentów wygenerowano za pomocą skryptu w języku `python`.

3.3. Zadanie 3 – semafor licznikowy

Semafor licznikowy zaimplementowano w klasie `CounterSemaphore` (fragment kodu 4) wyłącznie z użyciem dwóch semaforów binarnych:

- `mutex` (początkowo otwarty) – chroni pola `value` (aktualna wartość semafora) i `waiting` (liczba wątków czekających),
- `delay` (początkowo zamknięty) – na nim blokują się wątki, które muszą czekać, gdy `value == 0`.

Zastosowano technikę przekazywania pałeczki (*baton passing*): wątek budzony przez `delay.V()` dziedziczy własność `mutex`, więc między obudzeniem a zmniejszeniem `value` żaden inny wątek nie może „ukraść" zasobu. Metoda pomocnicza `signal()` albo budzi kolejny czekający wątek (jeśli `waiting > 0` i `value > 0`), albo zwalnia `mutex`.

```
1  class CounterSemaphore {
2      int value = 0;
3      Semaphore mutex = new Semaphore();
4      Semaphore block = new Semaphore();
5
6      public CounterSemaphore(int value) {
7          this.value = value;
8          if (value == 0) { this.block.P(); }
9      }
10
11     public synchronized void P() {
12         block.P();
13         mutex.P();
14         value--;
15         while (value > 0) { block.V(); }
16         mutex.V();
17     }
18
19     public synchronized void V() {
20         mutex.P();
21         value++;
22         if (value == 1) { block.V(); }
23         mutex.V();
24     }
25 }
```

Fragment kodu 4: Plik `CounterSemaphore.java` – semafor licznikowy z semaforów binarnych

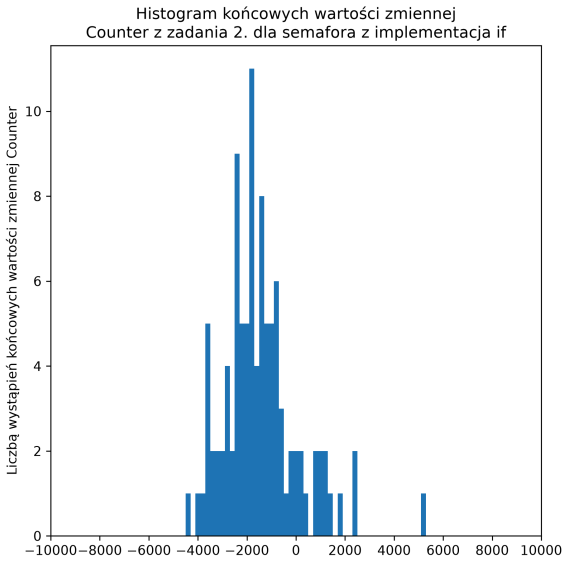
4. Wyniki doświadczenia

4.1. Zadanie 1 i 2

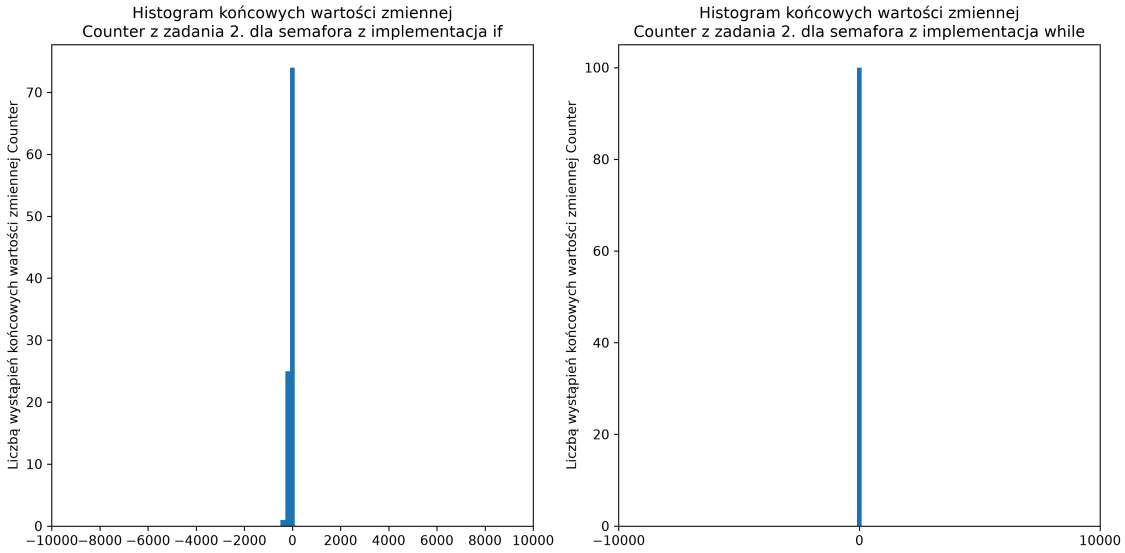
Statystyki końcowych wartości zmiennej `Counter` bez synchronizacji oraz z wykorzystaniem semaforów zestawiono w tabeli 1. Histogramy wartości zmiennej `Counter` bez synchronizacji przedstawiono na rysunku 1, a z synchronizacją semaforami na rysunku 2.

Statystyka	**Bez synchronizacji**	**Semafor z `if`**	**Semafor z `while`**
Liczba wykonań	100	100	100
Wykonania z wynikiem 0	0	45	100
Minimum	−4382	−239	0
Maksimum	5312	81	0
Średnia	−1339,29	−0,47	0,0
Mediana	−1574,0	0,0	0,0

Tabela 1: Statystyki końcowych wartości zmiennej `Counter`



Rysunek 1: Histogram wartości końcowych zmiennej `Counter` programu bez synchronizacji



Rysunek 2: Histogramy wartości końcowych zmiennej `Counter` programów synchronizowanymi semaforami

W wersji bez synchronizacji wyniki są rozproszone, co potwierdza występowanie wyścigu. Semafor z pętlą `while` dał w każdym z 100 wykonań wynik 0, więc poprawnie realizuje wzajemne wykluczanie. Semafor z instrukcją `if` w sytuacji, gdy z semafora korzysta więcej niż dwa wątki, nie gwarantuje wzajemnego wykluczania: część wykonań kończy się wynikiem różnym od 0.

4.2. Wyjaśnienie teoretyczne

Z każdym obiektem w Javie związane są dwie kolejki:

- **kolejka wejścia do monitora** (*entry set*) – wątki chcące zdobyć zamek obiektu (wejść do metody `synchronized`),
- **kolejka oczekujących** (*wait set*) – wątki, które wywołały `wait()`, zwolniły zamek i czekają na `notify()`.

Wywołanie `notify()` nie przekazuje zamka obudzonemu wątkowi. Wątek ten jedynie przechodzi z kolejki oczekujących do kolejki wejścia do monitora i musi konkurować o zamek z innymi wątkami. Przeplot prowadzący do błędu dla `SemaphoreIf` (wątki A, B; semafor początkowo otwarty):

1. wątek A wywołuje `P()` i wchodzi do sekcji krytycznej (`open = false`),
2. wątek B wywołuje `P()`, widzi `open == false` i wykonuje `wait()` – trafia do kolejki oczekujących,
3. wątek A wywołuje `V()`: ustawia `open = true` i `notify()` – wątek B przechodzi do kolejki wejścia do monitora, ale zamka jeszcze nie ma,
4. zanim B zdobędzie zamek, wątek A (który właśnie wywołał `P()`) zdobywa zamek, widzi `open == true`, ustawia `open = false` i wchodzi do sekcji krytycznej,
5. wątek B zdobywa zamek i wznawia działanie **za** instrukcją `wait()`. Przy `if` nie sprawdza ponownie warunku, ustawia `open = false` i również wchodzi do sekcji krytycznej.

Pętla chroni także przed tzw. *spurious wakeups*, czyli obudzeniem wątku bez wywołania `notify()`, które specyfikacja Javy dopuszcza.

4.3. Zadanie 3

Semafor binarny jest szczególnym przypadkiem semafora ogólnego. Semafor ogólny o wartości początkowej 1 (lub 0), w którym operacje P i V występują naprzemiennie, zachowuje się jak semafor binarny – jego wartość nie przekracza 1. Różnica pojawia się przy niepoprawnym użyciu: w semaforze binarnym kolejne `V()` na otwartym semaforze nie mają dodatkowego efektu (wartość pozostaje 1), a w semaforze ogólnym zwiększają wartość ponad 1 i pozwalają później kilku wątkom wejść do sekcji. Semafor ogólny ograniczony od góry do 1 jest równoważny semaforowi binarnemu.

5. Wnioski

Implementacja semafora za pomocą `wait` i `notify` wymaga sprawdzania warunku w pętli `while`. Instrukcja `if` zakłada, że obudzony wątek natychmiast po `notify()` zastaje stan taki jak w chwili powiadomienia, a tak nie jest, ponieważ obudzony wątek musi jeszcze odzyskać zamek i może zostać wyprzedzony przez inny wątek.

Semafor licznikowy da się zbudować z semaforów binarnych, ale wymaga to staranności (technika przekazywania pałeczki): naiwne rozwiązania z jednym semaforem `delay` mogą gubić sygnały lub dopuszczać wyprzedzanie wątków. Semafor binarny jest szczególnym przypadkiem semafora ogólnego, gdy jego wartość jest ograniczona do 0 i 1.

Błędy synchronizacji są niedeterministyczne i ujawniają się tylko w określonych przeplotach.

Bibliografia

[1] GeeksforGeeks, „Binary Semaphore”. Dostęp: 18 października 2026. [Online]. Dostępne na: https://www.geeksforgeeks.org/operating-systems/binary--semaphore-in-operating-system/

[2] GeeksforGeeks, „Difference Between Counting and Binary Semaphores”. Dostęp: 18 października 2026. [Online]. Dostępne na: https://www.geeksforgeeks.org/operating-systems/difference-between-counting-and-binary-semaphores/

[3] „Show how counting semaphores can be implemented using only binary semaphores and …”. Dostęp: 18 października 2026. [Online]. Dostępne na: https://stackoverflow.com/questions/15117155/show-how-counting--semaphores-can-be-implemented-using-only-binary-semaphores-and